

Aplicativo Android que armazena e sincroniza perfis de equalização de som

Relatório de implementação de aplicativo Android para gerenciamento de perfis de audio.

CURSO

FORMAÇÃO EM SISTEMAS EMBARCADOS

LEANDRO MENDES DOS SANTOS

Links

Link do Repositório: [gitHub](#)

Link da Aplicação: [VehicleEqualizerApp](#)

Objetivo

- Explicar o ciclo de vida das Activities;
- Configurar corretamente as permissões de segurança no arquivo Android Manifest;
- Utilizar princípios de Programação Orientada a Objetos no desenvolvimento Android;
- Integrar componentes de interface gráfica com a classe Java ou Kotlin;
- Configurar Intents para navegação entre Activities.

Introdução

Este documento é uma atualização do documento 1 e tem por objetivo relatar todas as melhorias implementadas no aplicativo. A primeira melhoria foi a mudança da ListView para a RecyclerView, a segunda mudança foi em alguns aspectos da interface e, por fim, a implementação de um banco de dados local com Room além da atualização dos testes unitários.

Sumário

1. [Mudança da ListView para a RecyclerView](#)
 - 1.1 [Mudanças no Código](#)
2. [Mudança na interface](#)
3. [Implementação de um Banco de Dados Local Com Room](#)
 - 3.1 [Dependências](#)
 - 3.2 [Entidade](#)
 - 3.3 [Data Access Object](#)
 - 3.4 [Database](#)
 - 3.5 [Repository](#)
 - 3.6 [ViewModel](#)
4. [Atualização dos Testes Unitários](#)
5. [Referências](#)

1. Mudança da ListView para a RecyclerView

O que motivou a mudança na estrutura de visualização dos perfis para o **RecyclerView** foi que a sua principal vantagem sobre o **ListView** é a sua eficiência superior e flexibilidade no tratamento de grandes listas.

A tabela a seguir resume os pontos-chave que me fizeram realiza a migração[1].

Vantagem	RecyclerView (Padrão Atual)	ListView (Legado)
Performance e Memória	Padrão ViewHolder obrigatório por design, garantindo a reciclagem de Views e rolagem suave desde o início.	O padrão ViewHolder era opcional ; a ausência dele causava lentidão e alto consumo de memória.
Flexibilidade de Layout	Usa LayoutManager para suportar facilmente listas verticais, horizontais, grade (Grid) ou escalonadas (Staggered) .	Limitado a listas verticais. Para outros layouts, era necessário usar componentes separados (GridView).
Animações	Suporte nativo para animações de item (adição, remoção, movimento) via ItemAnimator , oferecendo uma UX dinâmica.	Não possuía suporte nativo, exigindo soluções complexas e manuais.
Atualização de Dados	Notificações granulares (notifyItemInserted, notifyItemRemoved, etc.) e suporte a DiffUtil , otimizando o redesenho apenas para itens alterados.	Geralmente usava notifyDataSetChanged() , que redesenhava toda a lista a cada alteração, sendo ineficiente.
Decoração de Itens	Fácil de adicionar divisores e decorações complexas usando ItemDecoration .	Dificuldade em personalizar divisores ou adicionar outras decorações.

1.1 Mudanças no Código

Essa é a estrutura de pastas atual:

```
.
├── com
│   └── leandromendes
│       └── vehicleequalizer
│           ├── data
│           │   ├── AppDatabase.kt
│           │   ├── model
│           │   │   └── EqualizerProfile.kt
│           │   ├── ProfileDao.kt
│           │   └── repository
│           │       └── UserRepository.kt
│           ├── MainActivity.kt
│           ├── ProfileApplication.kt
│           ├── ui
│           │   ├── EqualizerActivity.kt
│           │   └── viewmodel
```

```
├── MainViewModelFactory.kt
├── MainViewModel.kt
└── util
    ├── Constants.kt
    └── ProfileRecyclerViewAdapter.kt
```

A primeira alteração foi no arquivo `build.gradle.kts` adicionando a dependência `libs.androidx.recyclerview`, conforme abaixo:

```
implementation(libs.androidx.recyclerview)
```

A segunda alteração foi a implementação da classe `ProfileRecyclerViewAdapter` que estende a `RecyclerView.Adapter`. O `ProfileRecyclerViewAdapter` é a ponte que liga a lista de dados (no nosso caso a `EqualizerProfile`) ao `RecyclerView`. Sua função central é gerenciar os dados, criar as Views necessárias (`onCreateViewHolder`), e conectar os dados aos `ViewHolders` para reutilização (`onBindViewHolder`), além de fornecer um método para atualizar a lista.

Outra mudança é que essa classe agora configura os listeners de clique.

Por fim, sua inicialização na `MainActivity`.

```
// Initialize adapter for profile list
val recyclerView = findViewById<RecyclerView>(R.id.profileList)
recyclerView.layoutManager = LinearLayoutManager(this)
```

2. Mudança na interface

As duas imagens abaixo apresentam a versão antiga e a atual da tela inicial com a lista de perfis.

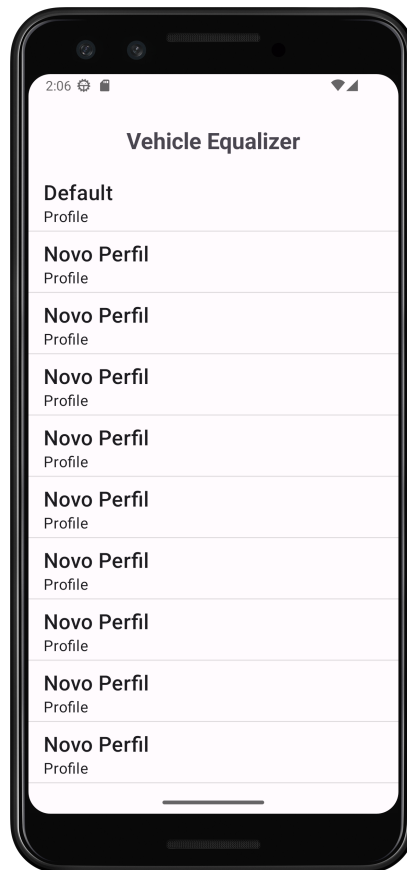


Figura 1: Tela inicial com vários perfis, na versão antiga

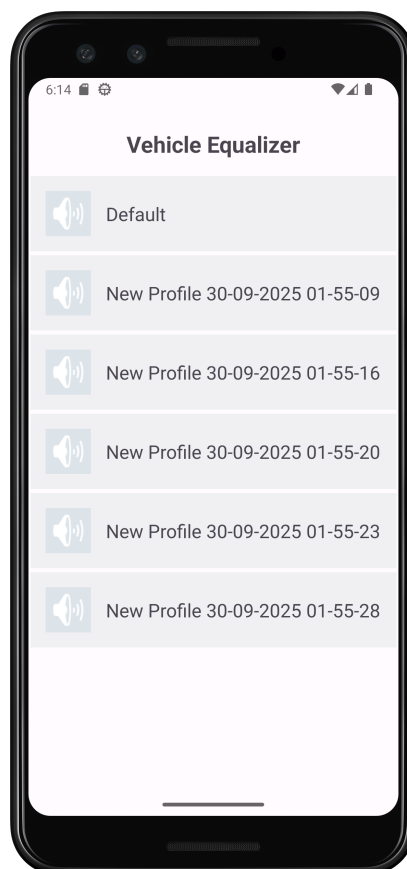


Figura 2: Tela inicial com vários perfis, na nova versão

A mudança, apesar de simples, permite que no futuro deva ser possível a escolha de alguma imagem associada ao perfil de áudio escolhido.

3. Implementação de um Banco de Dados Local Com Room

Para o armazenamento de dados dos perfis no meu aplicativo, escolhi utilizar o banco de dados de armazenamento local do tipo Room Database. Apesar de terem sido necessárias várias mudanças no código para a implementação, o uso do Room é uma solução bastante robusta e eficiente no contexto local[2].

Abaixo, vou descrever cada uma das alterações feitas no código para implementação do armazenamento local com Room Database[3].

3.1. Dependências

Adicionando as bibliotecas do Room no `build.gradle (app)` [4]

Essa parte é relativamente simples, basicamente foram adicionados as dependências a seguir e feito o sync da aplicação.

- `build.gradle (app)`

```
dependencies {  
  
    ...  
  
    implementation(libs.androidx.room.runtime)  
    implementation(libs.androidx.room.ktx)  
    implementation(libs.androidx.lifecycle.viewmodel.ktx)  
    ksp(libs.androidx.room.compiler)  
  
}
```

Como o código está sendo implementado em Kotlin, é necessário adicionar o KSP[5] (Kotlin Symbol Processing).

- `build.gradle (Project)`

```
plugins {  
    ...  
    id("com.google.devtools.ksp") version "2.0.21-1.0.27" apply false  
}
```

3.2. Entidade (Entity)

Mapeamento da classe `EqualizerProfile` como uma entidade Room.

As duas principais mudanças na classe foram[6][7]:

- Adição da anotação `@Entity` e define o nome da tabela;
- Adicionar o comando `id` e definir o ID como chave primária e auto-gerada.

Resultado:

```
...

@Entity(tableName = "equalizer_profiles")
@Parcelize
data class EqualizerProfile(
    // Sets the ID as the primary key and auto-generated
    @PrimaryKey(autoGenerate = true)
    val id: Int = 0, // New ID field for Room
    ...
) : Parcelable
```

A entidade representa o objeto que vai ser armazenado[8], que no meu caso são os perfis de equalização.

3.3. DAO (Data Access Object)

Criação de uma interface para definir as operações de CRUD (inserir, ler, atualizar, excluir).

Para a implementação do DAO foi criado o arquivo `ProfileDao.kt`. O arquivo fornece os métodos que o restante do app usa para interagir com os dados na tabela da entidade mostrada anteriormente[9][10][11].

Resultado:

```
... Imports
@Dao
interface ProfileDao {

    // Returns LiveData so that the UI can observe changes in real time
    @Query("SELECT * FROM equalizer_profiles ORDER BY id ASC")
    fun getAllProfiles(): LiveData<List<EqualizerProfile>>

    // Inserts a new profile. If there is a conflict, it replaces it
    @Insert(onConflict = OnConflictStrategy.Companion.REPLACE)
    suspend fun insert(profile: EqualizerProfile): Long // Returns the ID
    of the new item

    // Updates an existing profile
    @Update
    suspend fun update(profile: EqualizerProfile)

    // Remove a profile
    @Delete
    suspend fun delete(profile: EqualizerProfile)

    // Remove a profile by ID
    @Query("DELETE FROM equalizer_profiles WHERE id = :profileId")
    suspend fun deleteById(profileId: Int)
```

```
}
```



Nota Para mais detalhes sobre os métodos de conveniência utilizados, veja [Métodos de conveniência\[12\]](#).

3.4. Database

Criação da classe **RoomDatabase** para ser o ponto de acesso ao banco.

O código abaixo mostra a classe abstract chamada **AppDatabase**. A sua função é armazenar o banco de dados e definir a configuração do banco de dados além de servir como o ponto de acesso principal do app aos dados persistidos[13].

```
... Imports

// Defines the list of entities (EqualizerProfile) and the database version
(1)
@Database(entities = [EqualizerProfile::class], version = 1, exportSchema =
false)
abstract class AppDatabase : RoomDatabase() {

    // Method to obtain the Data Access Object for EqualizerProfile
    abstract fun profileDao(): ProfileDao

    companion object {
        @Volatile
        private var INSTANCE: AppDatabase? = null

        fun getDatabase(
            context: Context,
            scope: CoroutineScope // Scope required for asynchronous
database initialization
        ): AppDatabase {
            // Creates the database if INSTANCE is null (synchronized
block)
            return INSTANCE ?: synchronized(this) {
                val instance = Room.databaseBuilder(
                    context.applicationContext,
                    AppDatabase::class.java,
                    "equalizer_database"
                )
                    .fallbackToDestructiveMigration(false)
                    // Adds a callback to initialize the default profile
                    .addCallback(ProfileDatabaseCallback(scope))
                    .build()
                INSTANCE = instance
                instance
            }
        }
    }
}
```

```

        // Callback to populate the database the first time with the default
        profile
        private class ProfileDatabaseCallback(
            private val scope: CoroutineScope
        ) : Callback() {
            override fun onCreate(db: SupportSQLiteDatabase) {
                super.onCreate(db)
                INSTANCE?.let { database ->
                    scope.launch(Dispatchers.IO) {
                        // Creates and inserts the default profile only once
                        when creating the database
                            database.profileDao().insert(EqualizerProfile())
                    }
                }
            }
        }
    }
}

```

Como mostrado na primeira versão do App, um perfil chamado perfil Default sempre está presente e não pode ser apagado ou modificado, sendo assim foi implementado uma callback que povoa o banco de dados com o perfil padrão caso o banco esteja vazio[14].

3.5. Repository

Atualizar `UserRepository.kt` para usar o DAO em vez de uma lista em memória como era no App original.

Principais alterações no arquivo:

```

... Imports

class UserRepository(private val profileDao: ProfileDao) {
    // Returns LiveData of profiles. This list will be used by the
    ViewModel.
    val allEqualizerProfiles: LiveData<List<EqualizerProfile>> =
    profileDao.getAllProfiles()

    suspend fun addProfile(equalizerProfile: EqualizerProfile) {
        // ID is 0 (autoGenerate) so that Room inserts it as new
        profileDao.insert(equalizerProfile.copy(id = 0))
    }

    suspend fun removeProfile(profile: EqualizerProfile) {
        profileDao.delete(profile)
    }

    suspend fun updateProfile(equalizerProfile: EqualizerProfile) {
        // Assume that the EqualizerProfile object already has the database
        ID
        profileDao.update(equalizerProfile)
    }
}

```



```
}  
}
```

A partir do Room 2.1 é possível usar a palavra-chave `suspend` para tornar as consultas do DAO assíncronas[15].

3.6. ViewModel

Adaptação da `MainViewModel.kt` para usar a nova assinatura do `Repository` e retornar `LiveData` para observação assíncrona.

`MainViewModel.kt`

Principais alterações feitas na classe `MainViewModel.kt`:

1. Receber o `Repository`.
2. Retornar o `LiveData<List<EqualizerProfile>>` do Room.
3. Executar as operações de banco em uma coroutine (usando `viewModelScope`).

```
... Imports  
  
// MainViewModel now receives the Repository in the constructor  
class MainViewModel(private val userRepository: UserRepository) :  
    ViewModel() {  
  
    // LiveData that MainActivity will OBSERVE  
    val allProfilesLiveData: LiveData<List<EqualizerProfile>> =  
        userRepository.allEqualizerProfiles  
  
    ... // Adaptação para o uso do viewModelScope  
  
    fun addProfile(equalizerProfile: EqualizerProfile) =  
        viewModelScope.launch {  
            userRepository.addProfile(equalizerProfile)  
        }  
  
    fun updateProfile(equalizerProfile: EqualizerProfile) =  
        viewModelScope.launch {  
            userRepository.updateProfile(equalizerProfile)  
        }  
  
    fun removeProfile(equalizerProfile: EqualizerProfile) =  
        viewModelScope.launch {  
            userRepository.removeProfile(equalizerProfile)  
        }  
  
    ...  
}
```

Para injetar o `UserRepository` corretamente com o `ProfileDao` e o `ApplicationContext`, foi necessário implementar `ViewModelProvider.Factory` customizada[16][17].

`MainViewModelFactory.kt`:

```
// Factory to instantiate MainViewModel with Repository
class MainViewModelFactory(private val repository: UserRepository) :
    ViewModelProvider.Factory {
    override fun <T : ViewModel> create(modelClass: Class<T>): T {
        if (modelClass.isAssignableFrom(MainViewModel::class.java)) {
            @Suppress("UNCHECKED_CAST")
            return MainViewModel(repository) as T
        }
        throw IllegalArgumentException("Unknown ViewModel class")
    }
}
```

Para iniciar o banco de dados e o repositório foi implementada a classe `ProfileApplication.kt`.

```
... imports

/**
 * Initializes important resources (database and repository) that need to
 * be
 * available throughout the application's lifetime.
 */
class ProfileApplication : Application() {
    // Use SupervisorJob so that a coroutine failure does not affect others
    val applicationScope = CoroutineScope(SupervisorJob())

    // Lazy initialization of the database and repository
    val database by lazy { AppDatabase.getDatabase(this, applicationScope)
}
    val repository by lazy { UserRepository(database.profileDao()) }
}
```

Adição no `AndroidManifest.xml`

```
...
    <application
        ...
        android:name=".ProfileApplication">
...

```

A classe `ProfileApplication` serve como classe base para manter o estado global do aplicativo e manipular tarefas de inicialização que precisam ocorrer antes que quaisquer outros componentes do

aplicativo (como Activities or Services) sejam criados[18].

Alterações feitas na MainActivity

1. Obter o ViewModel usando o Factory customizado.
2. Observar o LiveData de perfis.
3. Atualização da forma como interage com o Adapter.
4. Mudança da assinatura dos métodos de remoção/atualização do ViewModel.

... Primeira alteração

```
// Obtain the Application, Database, and Repository to inject into
the ViewModel
val application = application as ProfileApplication // Casting for
the new Application class
val factory = MainViewModelFactory(application.repository)
mainViewModel = ViewModelProvider(this, factory)
[MainViewModel::class.java] // Use this Factory
```

... Segunda alteração

```
// Observe the Room's LiveData and update the Adapter
mainViewModel!!.allProfilesLiveData.observe(this) { profiles ->
    // Updates the list in the Adapter and notifies the change
    currentProfileList = profiles // Updates the reference list
    (recyclerView.adapter as
ProfileRecyclerViewAdapter).updateProfiles(profiles)
    Log.d(logTAG, "Live Data profiles updated. Count:
${profiles.size}")
}
```

... Terceira mudança

```
// Initialize adapter for profile list
val recyclerView = findViewById<RecyclerView>(R.id.profileList)
recyclerView.layoutManager = LinearLayoutManager(this)

// Variable to store the current list of profiles.
// Initialized with an empty list. It will be filled by LiveData.
var currentProfileList: List<EqualizerProfile> = emptyList()
```

... Quarta mudança

```
// If the index received is -1, it means that it is a new
configuration,
// so it saves a new profile
if (position == Constants.define.NEW_PROFILE) {
    mainViewModel!!.addProfile(currentProfile)

    Log.d(logTAG, "Saving new profile")
} else {
    // Update uses the object ID.
    // The returned 'currentProfile' object already has the
```

```

database ID.
    mainViewModel!!.updateProfile(currentProfile)

    Log.d(logTAG, "Updating current profile")
}

```

Essas foram as principais mudanças implementadas no código para atender à biblioteca do Room conforme a figura abaixo.

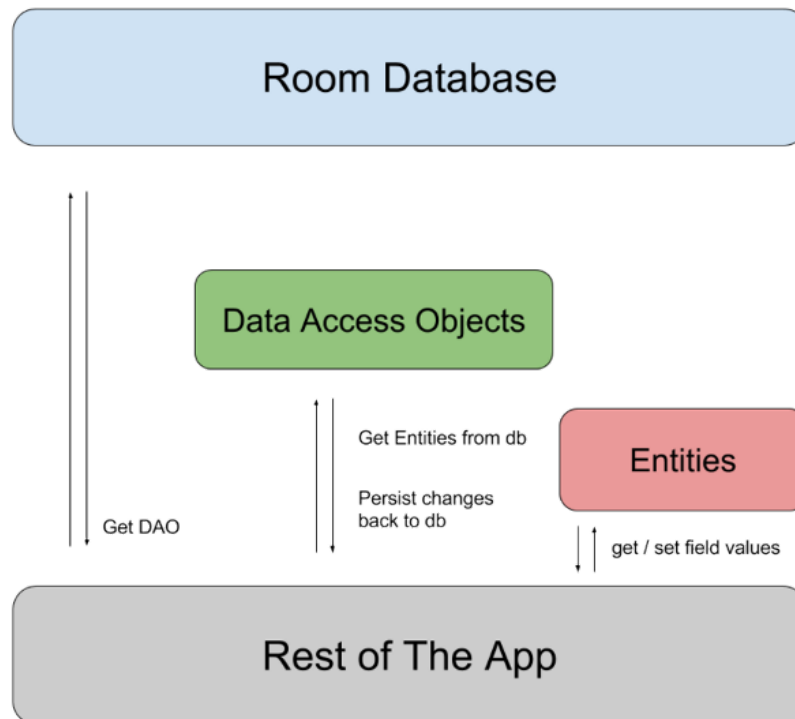


Figura 3: Diagrama da arquitetura da biblioteca do Room

4. Atualização dos Testes Unitários

O arquivo de teste unitário foi atualizado para atender as novas implementações feitas na classe X.

```

// --- LiveData Utility for Tests ---

/**
 * LiveData extension that blocks the test execution until a value is
 * emitted.
 */
fun <T> LiveData<T>.getOrAwaitValue(
    time: Long = 2,
    timeUnit: TimeUnit = TimeUnit.SECONDS,
    afterObserve: () -> Unit = {}
): T {
    var data: T? = null
    val latch = CountDownLatch(1)
    val observer = object : Observer<T> {
        override fun onChanged(value: T) {

```

```

        data = value
        latch.countDown()
        this@getOrAwaitValue.removeObserver(this)
    }
}
this.observeForever(observer)

try {
    afterObserve.invoke()

    // Don't wait indefinitely if the LiveData is not set.
    if (!latch.await(time, timeUnit)) {
        throw TimeoutException("LiveData value was never set.")
    }

} finally {
    this.removeObserver(observer)
}

@Suppress("UNCHECKED_CAST")
return data as T
}

// --- Fake DAO (Simulates Room) ---

/**
 * Fake/Mock implementation of ProfileDao for use in unit tests.
 */
class FakeProfileDao : ProfileDao {

    // Simulates the database table
    private val data = mutableListOf<EqualizerProfile>()
    // The MutableLiveData that notifies observers. Initializes with the
    // default profile.
    private val profilesLiveData = MutableLiveData<List<EqualizerProfile>>()

    ()
    private var nextId = 1

    init {
        // Initializes with the default profile
        val defaultProfile = EqualizerProfile(id = nextId++)
        data.add(defaultProfile)
        profilesLiveData.postValue(data.toList())
    }

    override fun getAllProfiles(): LiveData<List<EqualizerProfile>> {
        return profilesLiveData
    }

    override suspend fun insert(profile: EqualizerProfile): Long {
        // Copies the profile to ensure insertion uses a new ID
        val newProfile = profile.copy(id = nextId++)
        data.add(newProfile)
        profilesLiveData.postValue(data.toList())
    }
}

```

```

        return newProfile.id.toLong()
    }

    override suspend fun update(profile: EqualizerProfile) {
        val index = data.indexOfFirst { it.id == profile.id }
        if (index != -1) {
            data[index] = profile
            profilesLiveData.postValue(data.toList())
        }
    }

    override suspend fun delete(profile: EqualizerProfile) {
        if (data.removeIf { it.id == profile.id }) {
            profilesLiveData.postValue(data.toList())
        }
    }

    // Implementation required if it exists in ProfileDao, even if not used
    in tests
    override suspend fun deleteById(profileId: Int) {
        if (data.removeIf { it.id == profileId }) {
            profilesLiveData.postValue(data.toList())
        }
    }
}

@OptIn(ExperimentalCoroutinesApi::class)
class UserRepositoryTest {

    // Rule for LiveData to function correctly in the test
    @get:Rule
    var instantExecutorRule = InstantTaskExecutorRule()

    private lateinit var userRepository: UserRepository
    private lateinit var fakeProfileDao: FakeProfileDao
    private val testDispatcher = StandardTestDispatcher()

    private val defaultProfileName = Constants.define.PROFILE_DEFAULT_NAME
    private val newProfile1 = EqualizerProfile(
        name = "Profile_1",
        bassEqValue = Constants.define.BASS_VALUE_DEFAULT,
        midEqValue = Constants.define.MIDDLE_VALUE_DEFAULT,
        hiEqValue = Constants.define.TREBLE_VALUE_DEFAULT,
        balanceEqValue = Constants.define.PAN_VALUE_DEFAULT,
        masterVolValue = Constants.define.VOLUME_VALUE_DEFAULT,
        isSelected = false
    )

    @Before
    fun setUp() {
        // Sets up the Main dispatcher for coroutines
        Dispatchers.setMain(testDispatcher)

        // Initializes the Fake DAO and the Repository with the DAO (now

```

```

works)
    fakeProfileDao = FakeProfileDao()
    userRepository = UserRepository(fakeProfileDao)
}

@After
fun tearDown() {
    // Resets the Main dispatcher
    Dispatchers.resetMain()
}

// --- Initialization Tests ---
@Test
fun init_repositoryStartsWithOneDefaultProfile() = runTest {
    // WHEN: Gets the list of profiles from LiveData
    val profiles =
userRepository.allEqualizerProfiles.getOrAwaitValue()

    // THEN: The list size should be 1 and the name should be the
default
    assertEquals(1, profiles.size)
    assertEquals(defaultProfileName, profiles.first().name)
}

// --- Tests for addProfile ---
@Test
fun addProfile_addsNewProfileToListCorrectly() = runTest {
    // WHEN: Adds a new profile
    userRepository.addProfile(newProfile1)

    // THEN: The list size (LiveData) should be 2 and the new profile
should be at the end
    val profiles =
userRepository.allEqualizerProfiles.getOrAwaitValue()
    assertEquals(2, profiles.size)
    assertEquals(newProfile1.name, profiles.last().name)
}

// --- Tests for removeProfile ---
@Test
fun removeProfile_removesProfileByObject() = runTest {
    // GIVEN: Adds a second profile and saves the object for removal
    userRepository.addProfile(newProfile1)
    var profiles =
userRepository.allEqualizerProfiles.getOrAwaitValue()
    val profileToRemove = profiles.last()
    assertEquals(2, profiles.size)

    // WHEN: Removes the profile by object
    userRepository.removeProfile(profileToRemove)

    // THEN: The size should return to 1
    profiles = userRepository.allEqualizerProfiles.getOrAwaitValue()
    assertEquals(1, profiles.size)
}

```

```

        assertEquals(defaultProfileName, profiles.first().name)
    }

    // --- Updated Tests for updateProfile ---
    @Test
    fun updateProfile_replacesProfileAtValidIndex() = runTest {
        // GIVEN: Gets the default object and its ID.
        var profiles =
            userRepository.allEqualizerProfiles.getOrAwaitValue()
        val profileToUpdate = profiles.first()
        val originalId = profileToUpdate.id

        val newName = "Updated Profile"
        // Creates a new object with the same ID as the original
        val updatedProfile = profileToUpdate.copy(name = newName)

        // WHEN: Updates the profile
        userRepository.updateProfile(updatedProfile)

        // THEN: The profile name should be the new name and the size
        // should remain 1
        profiles = userRepository.allEqualizerProfiles.getOrAwaitValue()
        val updatedInList = profiles.first()

        assertEquals(newName, updatedInList.name)
        assertEquals(originalId, updatedInList.id)
        assertEquals(1, profiles.size)
    }
}

```

5. Referências

1. [Criar listas dinâmicas com o RecyclerView](#)
2. [Salvar dados em um banco de dados local usando o Room](#)
3. [Principais componentes \(Room\)](#)
4. [Configurar \(Room\)](#)
5. [Adicionar o plug-in KSP ao projeto](#)
6. [Entidade de dados](#)
7. [Anatomia de uma entidade](#)
8. [Definir dados usando entidades do Room](#)
9. [Objeto de acesso a dados \(DAO\)](#)
10. [Como acessar dados usando DAOs do Room](#)
11. [Anatomia de um DAO](#)

12. [Métodos de conveniência](#)
13. [Banco de dados](#)
14. [Preencher automaticamente o banco de dados do Room](#)
15. [Kotlin com fluxo e corrotinas](#)
16. [ViewModelProvider.Factory](#)
17. [Create ViewModels with dependencies](#)
18. [Customizing Your Android App with a Custom Application Class](#)